

Migration of X-Road to Post-Quantum Cryptography

Public Report

MTÜ Nordic Institute for Interoperability Solutions
Hobujaama 4, 10151 Tallinn, ESTONIA
Reg no 80419486

+372 7130 800
info@niis.org
www.niis.org





Nordic Institute for Interoperability Solutions (NIIS)

Hobujaama 4
10151 TALLINN
Estonia
+372 7130 800
info@niis.org
www.niis.org

Copyright © Nordic Institute for Interoperability Solutions (NIIS) 2025

ISBN 978-9916-9853-4-2 (pdf)
ISSN 2733-3345



Credits



UNIVERSITY OF TARTU

Institute of Computer Science

Author

University
of Tartu:

Dr. Mariia Bakhtina

Consultation

University
of Tartu:

Dr. Raimundas Matulevičius

NIIS:

Petteri Kivimäki



Table of contents

Credits.....	3
Glossary.....	5
Executive Summary.....	7
1. Introduction.....	8
2. Background.....	10
2.1. Symmetric and Asymmetric Cryptography.....	10
2.2. Quantum computing and PQC.....	12
2.3. Summary.....	13
3. Cryptographic Inventory for X-Road.....	14
3.1. Data in Transit.....	15
3.2. Data at Rest.....	16
3.3. Support Configurations.....	16
3.4. Summary.....	16
4. Quantum Computing Threats and Risks.....	18
4.1. Vulnerabilities.....	19
4.2. Threats of Quantum Computing (QC).....	20
4.3. Quantum Computing (QC)-based Risks to X-Road.....	22
4.4. Risks Assessment.....	23
4.5. Summary.....	24
5. Solutions to Mitigate QC-enables Threats and Risks.....	25
5.1. General mitigation measures.....	25
5.2. Mitigation measures of QC-enabled risks in X-Road.....	27
5.3. Summary.....	28
6. Implications of migration.....	30
6.1. Threats of Post-Quantum Cryptography.....	30
6.2. Challenges of PQC Implementation.....	31
6.3. Summary.....	32
7. Conclusion.....	33
References.....	34
A. Cryptographic Inventory of X-Road.....	37
B. Cypher suite in X-Road.....	51
C. Risks to X-Road Enabled by Quantum Computing.....	53
D. Risk Assessment Results.....	62



Glossary

Abbreviation	Description
AES	Advanced Encryption Standard
DH	Diffie-Hellman (key exchange)
DLP	Discrete logarithm problem
ECC	Elliptic Curve Cryptography
ECDH	Elliptic Curve Diffie-Hellman
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
eIDAS	electronic IDentification, Authentication and trust Services (regulation)
GDPR	General Data Protection Regulation (regulation)
GPG	GNU Privacy Guard
HKDF	HMAC-based Key Derivation Function
KEM	Key-Encapsulation Mechanism
IFP	Integer factorisation problem
OCSP	Online Certificate Status Protocol
PQC	Post-Quantum Cryptography



Abbreviation	Description
QC	Quantum computing
RSA	Rivest-Shamir-Adleman
TLS	Transport Layer Security



Executive Summary

For X-Road to maintain its foundational promise of a trustworthy digital ecosystem, a rigorous, forward-looking assessment is mandatory. To support NIIS in the migration of X-Road to post-quantum cryptography (PQC), this report presents the results of the X-Road study on the transition to quantum-resistant cryptography. The primary objective of this study is to study the migration process to PQC and risk assessment in the X-Road data exchange layer. As a result, this study aimed to identify the new threats that quantum computing brings to current cryptographic standards and how X-Road should be modified to protect against these threats.

The study is driven by the near-future availability of large-scale, fault-tolerant quantum computers (QC), which can leverage Shor's and Grover's algorithms to break current public-key infrastructure and significantly weaken symmetric key schemes. The urgency is driven by the "Harvest Now, Decrypt Later" threat, which exposes sensitive data with long retention periods to immediate compromise once the QC is implemented.

There are four key results of the study:

- **Cryptographic Inventory:** The research identified and catalogued 27 cryptographic assets within the X-Road data exchange layer, including keys, digital signatures, and hashes. This inventory provided the exact scope of assets vulnerable to quantum attack.
- **Threat and Risk Assessment:** A literature review resulted in the identification of 10 distinct QC-enabled threats. These threats were mapped to X-Road's cryptographic assets, defining a set of major QC-enabled risks (e.g., compromise of Message authenticity and long-term confidentiality) that pose a clear and present danger to the platform's security foundation.
- **Mitigation Strategy:** The primary defence against the QC-enabled threats is a planned, strategically prioritised transition to PQC. To manage risks during the transition, the report recommends implementing 11 specific security controls (categorised as preventative and corrective), which should be integrated into X-Road's architecture as soon as possible.
- **Implementation Challenges:** The migration will introduce new complexities, including performance overheads due to larger PQC keys and signature sizes, new algorithmic threats, and major logistical challenges in rebuilding the platform's Public Key Infrastructure.

The final conclusion and recommendations of this study are:

- The security risks posed by quantum computing to X-Road are serious and require an immediate strategic response. Postponing action will result in a significant loss of data security and operational assurance once functional quantum computers become available.
- It is strongly recommended that NIIS initiates a hybrid (dual-stack) PQC migration planning process immediately, focusing first on high-risk, long-confidentiality-life assets. The implementation of the 11 identified security controls must be prioritised to secure the platform during the multi-year transition period.

1. Introduction

X-Road is a centrally managed, distributed data exchange platform, designed to enable the standardised and secure provision and utilisation of web services within a trustworthy digital ecosystem [1]. This system fundamentally establishes an overarching framework that allows various organisations to interact and exchange information securely.

To ensure the integrity (with special attention to authenticity) and confidentiality of the data exchanged through X-Road, it employs hash functions and both symmetric and asymmetric cryptography. Meanwhile, the Shor's and Grover's algorithms prove the ability of quantum computing (QC) to weaken the security assumptions of traditional cryptography. Therefore, X-Road is prone to quantum threats, which should be mitigated.

Given the ongoing advancements by both prominent research institutions and private companies in developing fully working fault-tolerant quantum computers, their widespread availability is considered a matter of *when*, not *if*. This impending and inevitable threat necessitates a proactive global transition to *Post-Quantum Cryptography (PQC)*, which comprises novel cryptographic algorithms designed to withstand quantum-era attacks.

The urgency of this transition, often termed the “Y2Q problem” (Year-to-Quantum), is particularly acute for platforms like X-Road for two critical reasons:

- **“Harvest now, decrypt later” threat:** Adversaries are currently capable of harvesting large volumes of sensitive, encrypted X-Road traffic, storing it indefinitely, and decrypting it instantly once a functional quantum computer becomes available. This poses a severe risk to long-term confidentiality, especially for data with a retention requirement exceeding 10 years.
- **Migration complexity and time:** Cryptographic migrations are complex, expensive, and require significant lead time for planning, testing, and deployment. Since PQC involves mathematical foundations that in most cases differ from the ones used in classical cryptography, it necessitates system analysis, infrastructure upgrades, and a hybrid transition phase. Delaying the migration study until quantum computers are operational would leave the entire ecosystem vulnerable.

For X-Road to maintain its foundational promise of a trustworthy digital ecosystem, a rigorous, forward-looking assessment is mandatory. To support NIIS in the migration of X-Road to PQC, this study provides the mandatory initial analysis to ensure the security and longevity of the platform. The primary goal of our study is to examine the impact of quantum computing and cryptography on the X-Road data exchange layer. This study aims to identify the threats that quantum computing brings to X-Road and how X-Road should be changed to protect against these threats.

Report Overview. This report contains the results of a research study, and the rest of the report is organised as follows. Sect. 2 provides background of the study by describing the basics of hashing, symmetric and asymmetric cryptography, and the reasons why quantum computing is a threat to classical cryptography. Sect. 3 contains a cryptographic inventory of X-Road with the key cryptographic assets and algorithms used in the data exchange layer.



Sect. 4 describes vulnerabilities of classical cryptography, the threats and related risks which are enabled by quantum computing, based on the results from the literature review and cryptographic inventory. Specifically, Sect. 4.1 introduces vulnerabilities of cryptographic assets which are prone to quantum threats. Sect. 4.2 presents threats of quantum computing for the information systems in general, and Sect. 4.4 presents the QC-enabled risks to X-Road.

Sect. 5 introduces the mitigation measures to the QC-enabled risks, including migration to PQC and other mitigation strategies. Sect 6 provides an overview of threats which can be enabled by PQC and the potential challenges to be faced during migration to PQC. And, finally, Sect. 7 concludes the report with an overview of key results and recommendations for future work.

2. Background

This section provides the background knowledge necessary to understand the quantum threat to X-Road. It details the fundamental mathematical principles on which classical symmetric and asymmetric cryptography is based, explains the implications of quantum algorithms like Shor's and Grover's, and clarifies why the shift in computing capability makes the current digital security obsolete.

2.1. Symmetric and Asymmetric Cryptography

Cryptography provides the mechanisms for protecting information and communication against adversarial exploitation. Roughly, these mechanisms can be split into three groups – hash functions, symmetric cryptography, and asymmetric cryptography.

The *CIA Triad* (Confidentiality, Integrity, Availability) is a foundational model and guiding principle in information security. It consists of three core components that any security strategy aims to protect for an organisation's data and systems. *Confidentiality* refers to protecting information from unauthorised access or disclosure. *Integrity* refers to guarding against improper information modification or destruction, and ensuring data is accurate and trustworthy. Finally, *availability* refers to ensuring that systems and data are accessible and usable by authorised users when needed. The third security criterion, *availability*, is generally secured through system design, redundancy, and non-cryptographic controls, therefore, the availability criterion is left out of scope of this research.

As shown in Fig. 1, each cryptography mechanism has the security criteria from the CIA triad to be assured, namely, integrity and confidentiality. Additionally, some cryptography mechanisms target the security objective of *authenticity* as a specific subset of integrity. *Authenticity* refers to ensuring that a piece of information, a file, or a transaction came from the entity that claims to have sent or created it.

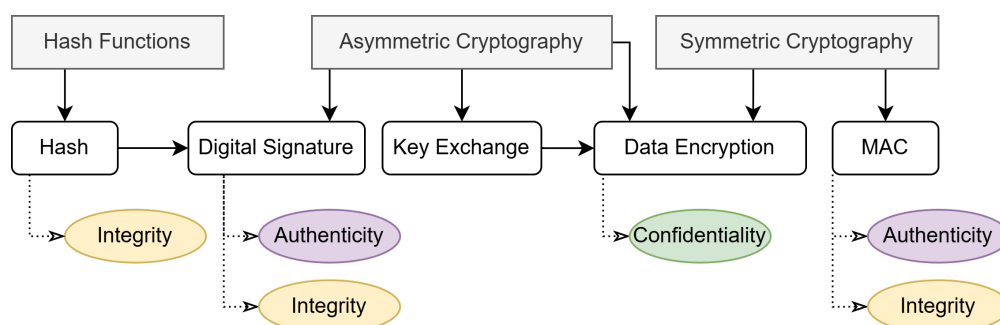


Figure 1: Cryptographic functions in traditional cryptosystems

Hash function refers to an algorithm for transforming the input data of arbitrary size into a hash value. The hash value is a fixed-size, one-way representation of input data that cannot be reversed to recover the original data. Hash functions do not necessarily require a cryptographic key to operate, but when a key is used (as in keyed hash functions), it is a symmetric key [2]. Due to the hash function's property of

being a one-way function, the hash is used for two purposes. First, hashes enable the verification of data integrity. By comparing a newly generated hash with a previously stored one, a user can detect if the underlying data has been altered. Second, hashes provide a layer of protection against disclosure through irreversibility. By storing a hash instead of the plaintext (e.g., common in scenarios like password storage) the system avoids retaining the original sensitive data. However, it is important to note that hashing does not provide formal confidentiality in the same way encryption does, as the original input may still be discovered via brute-force or dictionary attacks if the input space is predictable.

Encryption is a cryptographic technique of transforming readable information (i.e., plaintext) into an unreadable format (i.e., ciphertext) [3]. This transformation utilises *keys* as parameters for both encryption and decryption processes.

Symmetric cryptography refers to the algorithms of data protection of communication where both the sender and receiver of the message use one secret key. The *symmetric encryption* schemes use the same key for both encryption and decryption. Because they are computationally efficient, symmetric algorithms are the primary mechanism used (i) for ensuring the confidentiality of bulk data both in storage (i.e. data at rest) and (ii) for ensuring confidentiality of the shared data (i.e. data in transit). For securing data at rest, the key is managed locally by the data owner to prevent unauthorised access to storage. For securing data in transit, the key acts as a shared secret established between multiple parties to facilitate secure communication.

Security of the symmetric encryption relies on the secrecy of the key and the computational infeasibility of an exhaustive key search (brute-force attack) within a reasonable amount of time. The most common symmetric algorithm is *Advanced Encryption Standard (AES)*, which is a block cipher (i.e., encrypts data in fixed-size blocks). The security assurance offered by AES scales with its configurable key lengths of 128, 192, and 256 bits.

Message Authentication Code (MAC), also known as a cryptographic checksum, is a symmetric-key mechanism that provides message integrity and authentication by appending a fixed-length tag to a message. Unlike digital signatures, a MAC uses a shared secret key for both generation and verification, ensuring that a message originated from the correct sender and remained unaltered during transit. Because they are built upon block ciphers or hash functions (such as HMAC), MACs are significantly faster than asymmetric digital signatures, though they do not provide the service of non-repudiation.

Asymmetric cryptography (or “public key cryptography”) refers to the algorithms of data protection of communication where the sender and receiver use two different – namely, public key and private key. The private and public keys form a key pair, where each key is different, but they are mathematically connected. The private key should be kept secret by its holder, while the public key can be shared with anyone without compromising the security of the public key cryptography. In modern cryptographic systems, asymmetric cryptography is rarely used to encrypt the actual data directly. Instead, it is used to securely establish or transport the symmetric keys that perform the bulk data encryption for the data in transit. The asymmetric cryptography is mainly presented by algorithms for (i) asymmetric encryption, and (ii) digital signatures.

Asymmetric encryption uses a pair of keys for encryption and decryption to facilitate the secure exchange of information. While it can encrypt small amounts of data, in practice, algorithms like

Rivest-Shamir-Adleman (RSA), Elliptic Curve Cryptography (ECC), and RSA encryption is primarily used to instantiate Key Encapsulation Mechanisms (KEM) or to protect a symmetric session key. Asymmetric encryption relies on mathematical problems such as integer factorisation, discrete logarithm over multiplicative or additive groups that are computationally very difficult to solve for classical computers within a reasonable time frame.

Diffie-Hellman (DH) key exchange facilitates the secure establishment of a shared secret key between two communicating parties across insecure, public channels. The DH protocol leverages a private-public key pair for each participant to facilitate the secure derivation of a common secret, which is subsequently used as a symmetric encryption key to provide confidentiality for the actual data. While standard Diffie-Hellman (DH) operates based on modular exponentiation over large prime fields, *Elliptic Curve Diffie-Hellman (ECDH)* uses the distinct mathematical characteristics of elliptic curves. This allows ECDH to achieve comparable security with considerably smaller key sizes and offers enhanced computational efficiency.

Key-Encapsulation Mechanism (KEM) is the standard mechanism that facilitates the secure establishment of a shared secret in many modern and post-quantum protocols. Using the recipient's public key, the sender employs the KEM's encapsulation algorithm to generate both a compact, random secret key and an encrypted form (encapsulation or ciphertext) of that key. Subsequently, the receiver, possessing the corresponding private key, can utilise the KEM's decapsulation algorithm to retrieve the identical random secret key from the provided encapsulation. Such a secret key from the KEM is later used with symmetric encryption algorithms to encrypt the actual message or data.

A *digital signature* is a cryptographic technique used to verify the authenticity and integrity of digital messages or documents. Digital signatures guarantee that a digital message or document remains unchanged since its creation. This is achieved by generating a unique 'fingerprint' of the data, known as a signature. Also, some signature schemes use the hash-and-sign paradigm where instead of signing a message directly, the message is first hashed. Afterwards, the sender applies a cryptographic transformation to this hash using their private key by following the selected signing scheme. Because any alteration to the original data results in a completely different hash, any subsequent modification will invalidate the signature. While some legacy descriptions refer to this as "encrypting the hash", it is technically a distinct mathematical operation (i.e. signing). To verify the digital signature, the message receiver (i.e., verifier) uses the sender's public key and the agreed verification scheme (corresponding to the signing scheme). The most common digital signature schemes are *RSA*, and two ECC-based schemes, namely *Elliptic Curve Digital Signature Algorithm (ECDSA)* and *EdDSA (Edwards-curve Digital Signature Algorithm)*.

Assumptions and hard problems of cryptography. The security of cryptography relies on multiple assumptions. For the symmetric cryptography and hash functions, their security relies on the assumption that traditional computers have limitation of the processing time. Thus, the symmetric cryptography assumes that the search for a key to decrypt the ciphertext takes too much time (in a range of million of years) and power using the traditional computational powers. Therefore, assuming the symmetric key is kept safe, the security of symmetric ciphertext relies on *infeasibility to identify the secret key*. Similarly, the hash functions are one-way functions which rely on assumption that it is *not feasible to identify the original data input* based on the hash.

Meanwhile, the primary hard problems upon which the security of asymmetric cryptography schemes is based are integer factorisation problem and discrete logarithm problem:

- *Integer factorisation problem (IFP)*: This is the basis for algorithms like RSA. Its security relies on the practical impossibility of factoring very large composite numbers (products of two large prime numbers) into their prime factors [4, 5, 6, 7].
- *Discrete logarithm problem (DLP)*: This problem forms the basis for algorithms such as Diffie-Hellman (DH) and Elliptic Curve-based cryptosystems (ECC) such as ECDH, ECDSA, EdDSA, etc. For the cryptosystems based on the multiplicative groups, its security relies on the difficulty of finding the exponent x in the equation $g^x = y \pmod{p}$ given g , y , and p , where p is a prime number and g is what a generator of the group defined by p [4, 5, 6, 7].

2.2. Quantum computing and PQC

Quantum computing (QC) presents a profound challenge to contemporary cryptography. . Shor's Algorithm can solve the IFP and DLP in polynomial time. Since polynomial time is the standard for computational efficiency, this effectively renders current asymmetric schemes insecure.

Shor's algorithms show that factoring integers is efficient on an ideal quantum computer, and therefore, the existence of quantum computing eliminates the underlying assumption of the hardness of the integer factorisation problem [8, 9]. Thus, the algorithm poses a substantial threat to widely adopted cryptographic systems like RSA and ECC.

In contrast, **Grover's Algorithm** provides a quadratic speed-up for unstructured search attacks on symmetric-key cryptography [10, 6, 11]. Thereby, Grover's algorithm reduces the effective security level by half. While this is a significant acceleration, it remains in the exponential complexity class, meaning symmetric encryption can be made quantum-resistant by simply doubling the key length (e.g., moving from AES-128 to AES-256). As a result, QC presents a threat to *confidentiality* (via Shor's breaking of asymmetric encryption) and a significant reduction of security margin for *integrity* and *authenticity* (via Grover's attack on hashing/symmetric keys).

While both research institutions (like Max Planck Society, Massachusetts Institute of Technology and Chinese Academy of Sciences) and private companies (like IBM, Google and Baidu) work on the development of a fully functional, fault-tolerant quantum computer, it is only a matter of time before quantum computing is available. This inevitable threat necessitates the transition to Post-Quantum Cryptography (PQC), which encompasses new algorithms resistant to quantum attacks.

2.3. Summary

The background review established that X-Road's reliance on RSA/ECC for key exchange and digital signatures is vulnerable to Shor's polynomial-time attack, posing an existential threat to confidentiality and authenticity. Furthermore, Grover's algorithm reduces the effective security margin of symmetric



ciphers and hash functions, requiring an increase in key length to maintain current security levels. This justifies the necessity of an immediate transition to Post-Quantum Cryptography (PQC).

3. Cryptographic Inventory for X-Road

To accurately assess the quantum risk, this section systematically identifies and catalogues every cryptographic asset within the X-Road data exchange layer. The inventory examines cryptographic usage across data in transit, data at rest, and supporting configuration files to build a comprehensive security baseline.

The X-Road data exchange system employs cryptographic assets across its operational domains, ensuring the confidentiality, integrity, and authenticity of data both in transit and at rest, as well as the security of its support configurations. The inventory in this section describes the cryptographic assets, their functions, underlying protocols, and their integration with various X-Road components. The inventory is done based on version 7.6.2 of X-Road. The inventory is created by analysing the X-Road documentation and GitHub repository manually and using the Cryptographic Bill of Materials toolkit ¹ by IBM [12]. More details on the underlying protocols, together with the sources of the crypto inventory, and their integration with various X-Road features together with the sources of the crypto inventory can be found in Appendix A and Appendix B.

In total, we have identified 27 cryptographic assets represented by cryptographic keys, hashes, encrypted data entities and digital signatures. Figure 2 depicts a UML class diagram which illustrates the structural implementations of cryptographic data structures based on the identified assets and their dependencies with X-Road components.

¹ <https://github.com/IBM/cbomkit>

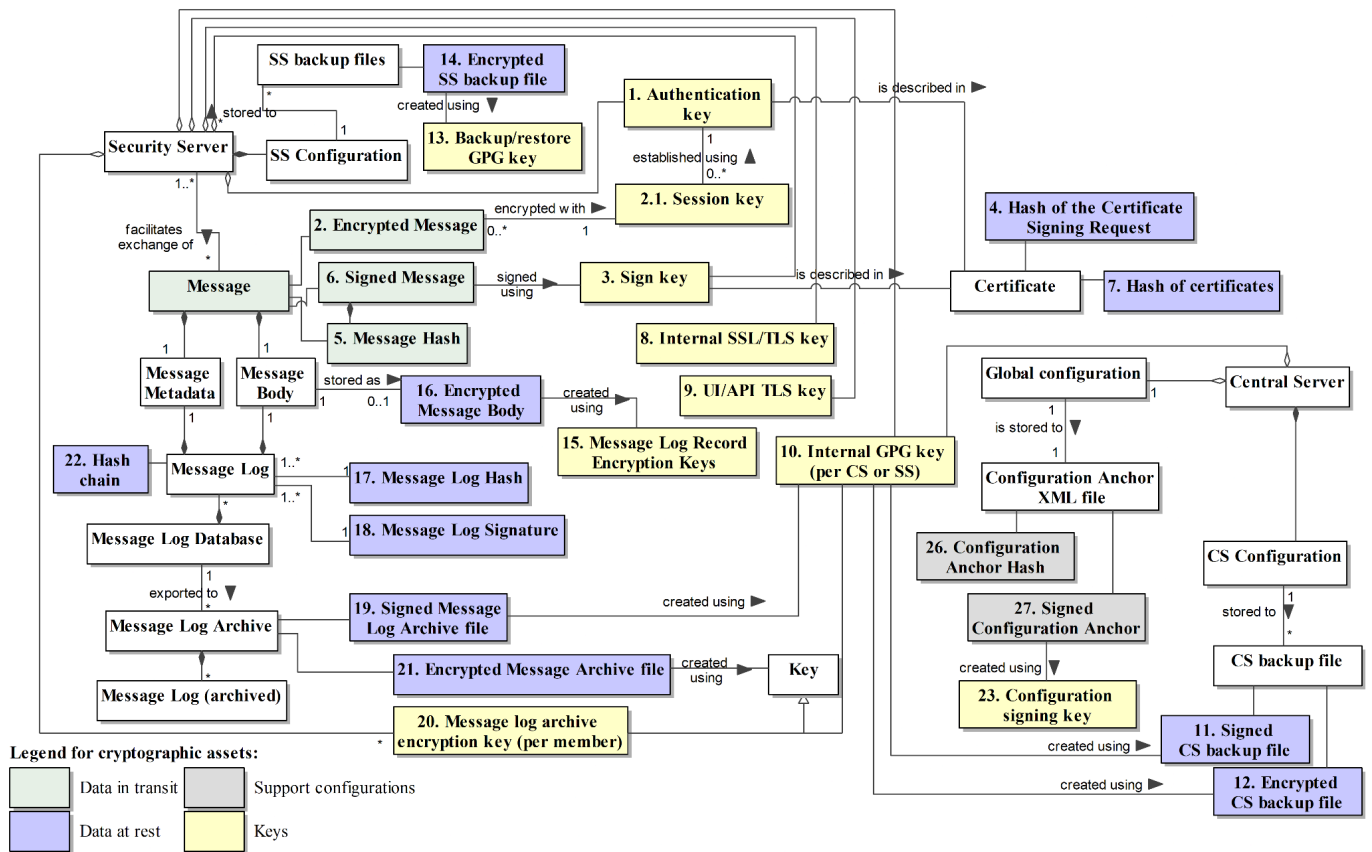


Figure 2: Cryptographic assets in X-Road

3.1. Data in Transit

For data exchanged between Security Servers, asymmetric cryptography is utilised for authentication, while the encryption uses symmetric cryptography. Authentication keys (RSA or ECC) and their associated certificates establish secure TLS connections for Security Servers. The session secret key is established by Security Servers during TLS handshake using Diffie-Hellman key exchange, where the Authentication key is used for encryption of session secret key. Then, Messages exchanged between Security Servers are encrypted using the session secret key. Furthermore, Sign keys (also RSA or ECC) are employed for generating digital signatures on messages (for the whole message including message body, metadata and attachments), commonly preceded by hashing the message with SHA-256. OCSP responses, which verify certificate status, involve SHA-1 hashes of certificates. Internal SSL/TLS keys secure communications between the Security Server and information systems, ensuring forward secrecy, server authentication, data confidentiality, and integrity. Additionally, UI/API TLS keys (2048-bit RSA) secure administrative interfaces.



3.2. Data at Rest

The protection of data stored within the X-Road ecosystem relies on a combination of asymmetric and symmetric encryption and digital signatures. Internal GPG keys on both Central and Security Servers are critical for signing, verifying, encrypting, and decrypting backup files and message log archives. Central Server backup files are signed and encrypted using these GPG keys. Security Server configuration backups can be encrypted with a block cipher (symmetric encryption), leveraging either the internal GPG key or administrator-defined additional GPG keys. If the message logging is enabled, Message log records utilise symmetric encryption with 128-bit keys derived via HMAC-based Extract-and-Expand Key Derivation Function (HKDF) with SHA-512, employing AES-128 in CTR mode for message body encryption within the message log database. Message log archive files, which are ZIP containers of signed and timestamped message logs, are signed using the internal GPG key (RSA) and can be encrypted with either the internal GPG key or member-specific keys. The integrity of message logs is further reinforced through a hash chain (Merkle tree) of messages which are created when the Message Log Database is exported to archived and are stored within the Message Log archives.

3.3. Support Configurations

Security of X-Road's configuration process is maintained through cryptographic controls. A configuration signing key (RSA and EC) generated by the Central Server administrator is used to sign the global configuration (RSASHA512). This ensures the authenticity and integrity of configuration data. Hashing mechanisms, including SHA512 (by default), SHA-256, and SHA-384, are utilised for configuration elements, ensuring their tamper-detection capabilities. The Configuration Anchor XML file contains the signed global configuration and public key of the Configuration Signing key.

It is worth noting that Appendix A contains assets 24 (Signature hash algorithm) and 25 (Hash algorithm) which are not depicted in Figure 2. While these are fundamental to X-Road's cryptographic operations, they represent abstract algorithm configurations and parameters rather than discrete data objects. Figure 2 is designed specifically as a class diagram to illustrate the structural implementation of cryptographic data structures. Assets 24 and 25 serve as the default algorithm settings defined within the X-Road configuration proxy, which are invoked when the system processes custom signatures or hashes. Consequently, because they function as procedural configurations rather than structural data entities, they are intentionally excluded from this architectural view to maintain the diagram's focus on implemented data objects.

3.4. Summary

The inventory reveals a strong reliance on established cryptographic algorithms and functions:



- Asymmetric Cryptography: Primarily RSA (2048-bit and 4096-bit) and Elliptic Curve Cryptography (ECC), specifically the secp256r1 curve, for authentication, key exchange, and digital signatures.
- Symmetric Cryptography: AES-128 and AES-256 in CBC and CTR modes for data confidentiality, particularly for internal server communications, data at rest and data in transit encryption.
- Hashing Functions: SHA-256, SHA-384, SHA-512, and SHA-1 (though SHA-1 is noted for OCSP requests, often considered a weaker hash for general-purpose security). HKDF with SHA-512 is used for key derivation.
- Protocols: TLS is fundamental for securing data in transit, employing various cipher suites to ensure security parameters.

In the next Section, we analyse the identified cryptographic assets on its vulnerabilities and define the QC-enabled risks.

4. Quantum Computing Threats and Risks

Building upon the inventory, this section translates the theoretical capabilities of quantum computing into a threat model for X-Road. It first establishes the cryptographic vulnerabilities of the identified assets, defines generalised quantum computing threats from a literature review perspective, and results in a detailed assessment of the resulting QC-enabled risks to the platform.

In this subsection, we discuss the threats of quantum computing both to cryptosystems and information systems. To create a threat model for X-Road, we first conduct a literature review. The literature review aims to answer the following research questions:

- RQ1: What are the threats of quantum computing to the information systems and data exchange systems?
- RQ2: What are the threats of PQC to information systems and data exchange systems?
- RQ3: What are the assets and their vulnerabilities targeted by the PQC and quantum computing threats?
- RQ4: What are the solutions for mitigating quantum computing- and PQC-based threats?

The targeted assets and their vulnerabilities, the threat of quantum computing and PQC and the solutions to mitigate those threats were identified through the literature review (LR) following the guidelines in [13]. The literature search was conducted using Scopus, IEEE Xplore and ScienceDirect. These databases were chosen because they include peer-reviewed papers and support the use of complex search queries with logical expressions. Additionally, we used Google Scholar for identifying the grey literature, as the topic is interesting for both academia and industry. For the literature review, we used the two queries:

- “quantum computing” AND (“threat” OR “risk”) AND (“information system” OR “information security”)
- (“post-quantum” OR “post quantum” OR “postquantum” OR “PQC”) AND “cryptograph*” AND “threat” AND (“information system” OR “information security”)

The search strategy resulted in 204 papers being found. Using the inclusion and exclusion criteria (depicted in Table 1), 21 papers were selected for the data extraction.

Table 1: Inclusion and exclusion criteria for the literature review

Exclusion Criteria (EC)	Inclusion Criteria (IC)
EC1: The paper is not digitally accessible.	IC1: The paper presents, reviews, discusses, or demonstrates a threat or risk of quantum computing or PQC to the information system.
EC2: The language of the paper is not English.	IC2: The paper describes at least one vulnerability or asset which is targeted by the quantum computing-based or the PQC-enabled attack.
EC3: The paper is a duplicate.	IC3: The paper describes at least one solution to mitigate quantum computing-based or PQC-based threats.
EC4: The paper is shorter than six pages.	

4.1. Vulnerabilities

Quantum computing poses a significant existential threat to current cryptographic standards by efficiently solving problems – namely, factorisation and discrete logarithm problems – that underpin widely deployed cryptosystems [4, 14, 11]. Table 2 depicts the security impact of quantum algorithms on cryptographic protocols themselves and the recommended mitigations by IBM Quantum [15].

As described in Table 2, quantum computing targets the vulnerabilities of the underlying cryptographic functions and their underlying security assumptions.

- **Vulnerability V1: Reliance on the factoring and integer factorisation problem.** Shor's algorithm proves the possibility of compromised asymmetric cryptography due to reliance on the integer factorisation problem. The potential functions prone to the V1 are *asymmetric encryption* and *digital signature algorithms*.
- **Vulnerability V2: Reliance on the discrete logarithm.** Shor's algorithm proves the possibility of compromised asymmetric cryptography due to reliance on the discrete logarithm problem. The potential functions prone to the V2 are *asymmetric encryption* and *digital signature algorithms*.

Table 2: Security impact of quantum algorithms on cryptographic protocols [15] and the related threats to information systems

Quantum Algorithm	Affected functionality or enabled attack (targeted vulnerability)	Impacted protocols	Related threats	Mitigation
Shor	Factoring (V1)	RSA	T1, T2, T5	Migrate to PQC
Shor	Discrete logarithm (V2)	Diffie-Hellman, DSA, Elliptic Curve Cryptography	T2, T5	Migrate to PQC
Grover	Key search (V3)	Symmetric key algorithms (e.g., AES)	T3, T4	Sufficient key length

Grover	Pre-image attack (V4)	Hash functions (e.g., SHA-256)	T6	Sufficient hash length
BHT	Collision attack (V5)	Hashfunctions (e.g., SHA-256)	T6	Sufficient hash length

• **Vulnerability V3: Reliance on the infeasibility of conducting a brute-force attack within a reasonable time frame.** Grover’s algorithm proves the possibility of conducting a brute-force search much faster by QC than by classical computers. The potential functions prone to the V3 are *symmetric key algorithms and hash functions*.

• **Vulnerability V4: Hash function’s inability to resist finding the original input of its one-way function (pre-image attack).** Grover’s algorithm proves the possibility of finding the original input of the hash function within a reasonable time frame due to the speed-up enabled by QC. The potential functions prone to the V4 are *hash functions*.

• **Vulnerability V5: Hash function’s inability to resist finding two different inputs that produce the same output hash value using a hash function (collision attack).** Grover’s algorithm proves the possibility of finding two different inputs that produce the same output hash value within a reasonable time frame due to the speed-up enabled by QC. The potential functions prone to the V5 are *hash functions*.

4.2. Threats of Quantum Computing (QC)

Table 3 summarises the threats identified through the literature review. Thus, the mapping highlights that the threats of quantum computing are defined by the targeted cryptographic asset – whether it is a (symmetric or asymmetric) key, (symmetrically or asymmetrically) encrypted data, digital signature or a hash.

Table 3: Threat of quantum computing on cryptographic assets

Cryptography type	Cryptographic asset	Threat ID	Threat description
Asymmetric cryptography	asymmetric key	–	
	RSA encrypted data	T1	Decrypt RSA-encrypted data
	digital signature	T2	Forge digital signatures
	asymmetrically encrypted data	T5	“Harvest Now, Decrypt Later”
Symmetric cryptography	symmetric key	T3	Brute-force attacks on symmetric encryption keys
	symmetric key with DH key agreement	T4	Compromise key agreement mechanisms
	symmetrically encrypted data	–	
Hash Function	hash	T6	Quantum-enhanced attacks on hash functions

Asymmetric Cryptography Weakening. Quantum computers, utilising algorithms such as Shor's, can drastically reduce the time required to compromise widely deployed asymmetric cryptosystems like RSA and Elliptic Curve Cryptography (including Elliptic Curve Diffie–Hellman (ECDH), Elliptic Curve Digital Signature Algorithm (ECDSA), and Edwards-curve Digital Signature Algorithm (EdDSA)) [4, 14, 11]. This reduction could be from billions of years to days or less [4]. ECC is identified as particularly vulnerable, requiring less quantum memory to break compared to RSA [4].

- **Decrypt RSA-encrypted data:** The usage of Shor's algorithm with a quantum computer would allow an adversary to directly derive the private key from a given RSA public key. With access to the private key, an attacker could then **decrypt any previously intercepted ciphertexts (T1)** encrypted with that public key or **forge digital signatures** attributed to the legitimate key holder. This capability would fundamentally undermine the security of widely deployed protocols, including TLS handshakes and VPN tunnels, potentially exposing sensitive data and compromising system integrity.

- **Forge Digital Signatures (T2):** RSA signatures, Digital Signature Algorithm (DSA), and ECDSA derive their security from the computational intractability of problems like integer factorisation or the discrete logarithm problem. A quantum adversary, utilising algorithms such as Shor's algorithm, could efficiently (in polynomial time) solve these underlying mathematical challenges. This capability would enable them to **forge digital signatures (T2)** that are computationally infeasible for classical computers to produce. As a result, this would compromise the authenticity and, therefore, the integrity of the used data.

Symmetric Cryptography Weakening. While symmetric cryptosystems like AES are generally considered more resilient, quantum computers can significantly reduce the complexity of key search operations, effectively weakening their security levels [4, 5, 6].

- Grover's Algorithm, for instance, can optimise **brute-force attacks on symmetric encryption keys (T3)**, thereby reducing the effective key length [14, 6]. For AES-128, the security level might be reduced from 128 to approximately 106 bits against quantum adversaries [4].

- As a consequence of T1, quantum computing capability extends to the ability to **compromise key agreement mechanisms (T4)** for symmetric keys (e.g., Diffie–Hellman key exchange), which are integral to protocols like TLS handshakes, through Shor's algorithm [16]. This capability poses a direct threat to the confidentiality of communications by allowing an eavesdropper to compute the shared secret key from the public values exchanged during the key establishment phase. Crucially, this means that these protocols would no longer provide forward secrecy: any past encrypted communications that were intercepted and recorded could be decrypted by recovering the ephemeral session keys, and ongoing communications could be subject to real-time man-in-the-middle (MitM) attacks as an adversary could derive the session keys [17].

"Harvest Now, Decrypt Later" Threat (T5). A critical concern is the "store now, decrypt later" attack strategy, where data encrypted using current cryptographic schemes may become vulnerable during its lifetime with the future advent of capable quantum computers [4, 18, 11]. The attack (also known as "Store Now, Decrypt Later" or "Harvest-Decrypt attack" [18]) is implemented by interception of encrypted data with the intention to store it by an unauthorised party. The attacker's strategy is to wait until the appearance of a working quantum computer that would allow them to decrypt the information using

Shor's algorithm, even without possessing the legitimate decryption key. This poses a significant risk to long-term confidentiality, particularly for sensitive information with enduring value [18, 19, 20].

Quantum-Enhanced Attacks on Hash Functions (T6). Hash functions are generally considered secure in the post-quantum era. However, the usage of Grover's algorithm can provide a quadratic speed-up for the brute force attacks on the hashed value to identify the original input [15]. For example, the currently used 256-bit hash function, which is considered secure against pre-image attack (i.e. identification of the original value from the hash value) by classical computers, would provide the same level of security as a 128-bit hash function when faced with a quantum attacker utilising Grover's algorithm [15].

4.3. Quantum Computing (QC)-based Risks to X-Road

The results of cryptographic inventory and the overview of X-Road functionality suggests that there are **6 business assets** in the system: Message (A1), Message Log (A2), Security Server Configuration (A3), Global Configuration (A4), Central Server Configuration (A5), and Member's Sign Key Certificate (A6). By applying the instantiating threats to the cryptographic assets in X-Road and considering these 6 business assets as ones that require assurance of confidentiality, integrity and authenticity, we have identified **32 security risks** which might be caused by QC. Appendix C contains the detailed risk descriptions. For each risk, we define the targeted business assets, related cryptographic assets, assured security criteria (integrity, confidentiality or authenticity), and the threat which enables the risk. Table 4 provides an overview of the risks targeting the business assets. Note that one risk may affect multiple security criteria, therefore, the number in R per each business asset is not equal to the sum of R_C , R_I and R_{Au} .

Table 4: Number of security risks per business asset and its affected security criteria (R_C - risks affecting confidentiality, R_I - risks affecting integrity, R_{Au} - risks affecting authenticity)

Business Asset	Overall No. of risks R	No. of risks R_C	No. of risks R_I	No. of risks R_{Au}
Message	13	10	3	1
Message Log	8	4	4	2
Security Server Configuration	3	3	0	0
Global Configuration	3	1	2	1
Central Server Configuration	2	1	1	1
Member's Sign Key Certificate	3	1	2	1

As shown in Table 4, most of the risks aim to target Messages or Message Logs, primarily affecting their confidentiality.

4.4. Risks Assessment

Estimate business asset value. To assess the priority of the identified security risks, we use the input of NIIS representatives to evaluate the level of business asset values (*BAL*) and the likelihood of QC-enabled threats (*ThL*). The *BAL* values are estimated by a value from 1 to 3 using the following scale:

- *BAL* = 1 (Low), if the compromise of the asset would result in minimal or negligible impact; the financial loss would be minor, reputational damage would be local, and business operations would be only briefly disrupted;
- *BAL* = 2 (Medium), if the compromise of the asset would cause noticeable impact; the financial loss would be moderate; there would be some regulatory or contractual violation; business operations would be degraded but can be recovered quickly, and there would be public or customer embarrassment;
- *BAL* = 3 (High), if the compromise of the asset would result in severe or catastrophic impact, a significant financial loss, major regulatory fines (e.g., GDPR), severe reputation damage leading to loss of market share, and complete or prolonged disruption of critical business functions.

Estimate threat likelihood. The threats likelihood values are estimated under assumption that the capabilities of quantum computers are more evolved compared to the time of the report preparation. Thereby, the values are likelihood estimates for the QC-enabled threats in a few years from now when the sufficiently powerful quantum computers are available. Analogously, the values of threats likelihood *ThL* are estimated by a value from 0 to 3 using the following scale:

- *ThL* = 0 (Unlikely), if the event is not expected to occur; it is improbable, and there are few historical precedents or current incentives for an attacker.
- *ThL* = 1 (Possible), if the event could occur at some point, but it is not a common occurrence; it may be due to a targeted or sophisticated attack.
- *ThL* = 2 (Likely), if the event is expected to occur at least once; there is a known vulnerability, a high-value target, and a motivated threat actor.
- *ThL* = 3 (Almost Certain), if the event will almost certainly occur, perhaps multiple times; there is an active, known exploit that is easily leveraged, or the threat is constant (e.g., phishing attempts).

Determine threat event potentiality. Risk event potentiality *EP* is calculated based on the metrics of threats that form a risk event. As there might be multiple threats leading to the same risk, the potentiality of the risks event depends on the threat likelihood and is calculated as maximum of likelihoods: $EP = \max(ThL_i)$, where ThL_i are threat likelihoods of each threat that can lead to the selected risk.

Determine risk impact. The risk impact *I* of the risk considers the value of affected asset and the violated security criteria of each business asset. Therefore, the risk impact is calculated as following: $I = BAL * N$, where *BAL* is the value of a business asset affected by the selected risk, and *N* is number of affected security criteria for the asset. In this report we assume that each risk affect only 1 asset.

Assess risk level. Having the metrics of all the risk components determined, we perform the following calculation to assess the risk level R : $R = EP * I$. Thereby, the risk level considers the potentiality of the risk event and the impact on the business assets it may cause. By this point, the risk assessment activity is finished. The risk assessment results can be found in Appendix D.

As the final step, we prioritise the risks based on their risk level. The risks with the risk level $R \in [1,4]$ are assigned low priority, the ones with the risk level $R \in [5,8]$ are assigned medium priority, and if $R \in [9,12]$ – the risk is a high priority. The results are depicted in Table 5.

Table 5: Risk assessment results

Priority	Risks
Low	R13, R14, R16, R21, R22, R23, R24, R25, R29, R30, R31
Medium	R5, R6, R9, R11, R12, R18, R19, R20, R26, R27, R28
High	R2, R10, R15, R17

As Table 5 shows, the risks R3, R11, R16, and R18 are assessed as high priority. The high priority risks are aiming to affect the high value business assets, namely Message and Message Log, are targeting both integrity and authenticity of the assets, and the underlying threats are of high likelihood to take place. In the next section we describe how the identified risks can be mitigated.

4.5. Summary

The analysis in this section established a chain of vulnerability to risk, confirming that all asymmetric cryptographic assets in X-Road are vulnerable to Shor's algorithm, threatening long-term confidentiality and authenticity of the exchanged data and the integrity of digital signatures. The section defined the state-of-the-art general quantum threats and concluded with an assessment of the QC-enabled risks and their prioritisation which serves as the baseline for the migration roadmap. In the next section we describe how the identified risks can be mitigated.

5. Solutions to Mitigate QC-enables Threats and Risks

This section focuses on providing actionable defences against the identified quantum risks. It first introduces general mitigation strategies for cryptographic systems before detailing specific, strategic controls tailored to the X-Road environment, with the ultimate goal of achieving a complete PQC migration.

5.1. General mitigation measures

The primary mitigation strategy against quantum threats involves the adoption of Post-Quantum Cryptography (PQC) where possible. Additionally, the presented below strategic migration measures can be involved.

PQC Algorithm Usage, Development and Standardisation. Efforts by organisations like NIST [21] are crucial for standardising PQC algorithms, which would support their wide usage. PQC algorithms are broadly categorised into lattice-based, hash-based, multivariate, code-based, and isogeny-based schemes [16, 5, 22]. Additionally, there are MPC-in-the-head-based signature which is based on secure multi-party computation [23]. Key candidates identified for standardisation include CRYSTALS-KYBER (key encapsulation mechanism) and CRYSTALS-Dilithium (digital signature), FALCON (digital signature), and SPHINCS+ (digital signature) [4, 14, 11, 16] and HQC (key encapsulation mechanism) [24, 4]. Round 4 of the NIST project is ongoing, involving other algorithms like BIKE, Classic McEliece, HQC, and SIKE [4]. The usage of PQC analogues instead of their traditional counterpart may help to mitigate threats T1, T2, and T4. Particularly, the usage of CRYSTALS-KYBER or HQC as KEM enables the avoidance of the threat T4, while CRYSTALS-Dilithium, FALCON, and SPHINCS+ as PQ digital signature schemes allow the avoidance of the threat T2.

Increased Symmetric Key Size. To counteract Grover's algorithm and mitigate the threat T3, the size of the symmetric key should be increased [6]. The National Security Agency (NSA) recommends using AES with key lengths of 192 or 256 bits for securing highly secure documents [5].

Increased Hash Size. To counteract Grover's algorithm and mitigate the threat T5, the output length of hash functions should be increased (e.g., twice) [5].

Hybrid Cryptosystems. A recommended strategy is the adoption of hybrid cryptosystems, which combine a PQC primitive with a well-studied traditional cryptosystem (e.g., RSA or ECDH) [4, 11, 20]. For example, hybrid cryptosystems for digital signature may include the usage of a composite signature composed of two parts – one is the signature created with the classical signature algorithm and another is the signature created with the post-quantum signature algorithm [4, 11, 20, 25]. Similarly, the hybrid KEM derives a final shared secret by concatenating outputs from both classical (e.g., ECDH) and

post-quantum algorithms [11]. This approach ensures security remains dependent on the stronger of the two components and facilitates migration by often being implementable as modifications to existing protocols like X.509 or TLS, offering backwards compatibility [4].

While all the ECC-based signatures are equally vulnerable to the quantum computing threats, the study in [26] highlights the advantage of EdDSA over ECDSA in the process of migration to PQC. Due to the deterministic nature of the key derivation in EdDSA, the private key created using EdDSA can be potentially used as a baseline for the PQC-based digital signature [26].

Prioritised Migration. Organisations should prioritise the migration of their most vulnerable information assets to a quantum-safe state, with elliptic curve cryptosystems (ECC) being recommended for replacement first due to their higher vulnerability [4, 20]. While in general RSA and ECC are equally vulnerable to QC, breaking ECC of the same level of security as RSA requires about 10 times more the quantum memory for the elliptic curve cryptosystems [4, 27] due to the shorter key size.

The migration of confidentiality mechanisms (to protect the data at rest) to PQC should have the highest priority. Governments, such as the US, are urging the implementation of post-quantum algorithms by 2035 to counter the "Harvest Now, Decrypt Later" threat (T5) [19]. On the other hand, the transition to PQC for the data at rest is considered easy. In contrast, migration of the protection of the data in transit is more complicated and resource-intensive due to the need for agreement between communicating parties and transition of cryptographic systems on both sides [20]. The PQC migration strategies, handbooks and checklists (e.g., [2, 28, 4, 29]) can be used by organisations for planning the migrations.

Industry Tools and Open-Source Initiatives to Support Migration to PQC. Various tools and initiatives are emerging to facilitate PQC adoption [11]. Examples include DigiCert's PQC toolkit [30], Sectigo's Quantum Safe Kit for quantum-safe and hybrid digital certificates [31], and Cloudflare's experiments with PQC primitives in TLS 1.3 [32]. The Open Quantum Safe project (OQS) provides an open-source C library (liboqs) for integrating PQC into existing protocols and applications like OpenSSL and OpenSSH [11].

Gradual Migration to PQC with Alternative PKI Certificates. To tackle PQC.T2 and enable gradual transition of systems to the PQC and PQC-based PKI, the alternative PKI certificates could be used [33]. Such alternatives as *hybrid*, *composite*, and *chameleon PKI Certificates* allow integrating post-quantum cryptography (PQC) into the existing certificate infrastructures. One of the ways to implement hybrid certificates is by using the ITU-T X.509 standard (the 2019 revision) [11, 34]. This revised standard now provides mechanisms for embedding alternative public key information, digital signature algorithms, and their corresponding signatures directly within a public key certificate. Specifically, these PQC elements can be integrated through the use of dedicated X.509 extensions: *subjectAltPublicKeyInfo*, *altSignatureAlgorithm*, and *altSignatureValue* [34]. As a result, parties capable of supporting PQC algorithms can leverage the alternative, quantum-resistant signatures, while those systems that have not yet transitioned can continue to rely on traditional cryptographic signatures. This ensures backwards compatibility during the migration period. However, for some systems, including these post-quantum elements in a certificate may lead to an increase in its overall size, which can have implications for network bandwidth, storage, and processing overhead in certain environments.

Architectural Adaptations. PQC needs to be integrated into various layers of data exchange systems. For example, within such systems like UXP and X-Road, the PQC migration should be implemented for (i) TLS connections between security servers and external clients, (ii) TLS connections between two security servers, (iii) message signing between security servers, (iv) global configuration signatures [11, 20], and (v) signing of timestamps.

Evidentiality Value Preservation. For digital signatures, steps must be taken to demonstrate that a signature was created before the emergence of a cryptographically significant quantum computer to preserve its evidential value [20].

5.2. Mitigation measures of QC-enabled risks in X-Road

Based on the general mitigation measures presented above, to address the security risks identified in Sec. 4.4, XRoad should implement the security controls which are either preventing the threats (P) or can correct the impact (C) of the associated risks. Table 6 presents 11 controls which can help mitigate QC-enabled risks in X-Road.

The mapping of these controls to the risks which they mitigate can be found in the last column in Table 7. The mitigation plan using the measures centers on a hybrid (dual-stack) deployment of PQC algorithms along with the traditional cryptography as the starting point of migration.

Table 6: Security controls to mitigate QC-enabled risks in X-Road

Addressed Threat	Requirement	Control	Reference
T1	X-Road should ensure confidentiality of the data in the context of QC.	C1.P. Use PQ encryption algorithm instead of RSA and ECC encryption for data at rest	[16, 5, 22]
		C2.P. Use TLS version with quantum-resistant components	[16]
T2	X-Road should ensure authenticity of the signed data in the context of QC.	C3.P. Use PQ digital signature algorithms (e.g., CRYSTALS-Dilithium, FALCON, SPHINCS+)	[4, 14, 11, 16, 21]
		C4.C. During the transition period, use Hybrid digital signature algorithms where the composite signature is created based on both classical and post-quantum signatures.	[4, 11, 20]
		C5.C. Use alternative PKI certificates for enabling gradual transition to PQ digital signature.	[33]



		C6.P. Adding the evidence of the DS creation time for the signatures created with traditional DS algorithms.	[20]
T3	X-Road should enable confidentiality of the used symmetric keys.	C7.C. Increase symmetric key size (e.g., to 192 or 256 bits for AES)	[5, 6]
T4	X-Road should ensure confidentiality of private keys established through key agreement in the context of QC.	C8.P. Use PQ key agreement mechanisms (e.g., CRYSTALS-KYBER, HQC)	[16, 5, 22, 21]
T5	X-Road should ensure confidentiality the stored encrypted data in the context of QC.	C1.P. Use PQ encryption algorithm (e.g, lattice-based, hash-based, multivariate, code-based, and isogeny-based) instead of RSA and ECC encryption for data at rest	[16, 5, 22]
		C9.P. Use PQ encryption algorithm (e.g, lattice-based, hash-based, multivariate, code-based, and isogeny-based) instead of ECC encryption for data at rest.	[16, 5, 22]
		C10.C. Avoid storage of the data of long-term value.	-
		C11.P. Increase output size of hash functions twice compared to the traditional security level standard.	[5]

5.3. Summary

The primary mitigation measure is the phased adoption of Post-Quantum Cryptography (PQC) candidates. Based on the assessed risks, the report recommends the implementation of 11 specific security controls (C1 to C11), categorised as either preventative (P) or corrective (C), to neutralise the threats. These controls represent a strategic roadmap for safeguarding X-Road's data integrity and confidentiality during the transition period.

6. Implications of migration

The final analytical section provides a forward-looking perspective on the migration process. While PQC solves the quantum problem, this section explores the new set of threats that PQC itself may introduce, alongside the significant operational and logistical challenges that must be addressed during the implementation phase.

6.1. Threats of Post-Quantum Cryptography

Increased Data Sizes (PQC.T1). This includes substantially larger sizes of PQC keys, ciphertexts, and signatures compared to their classical counterparts, which will necessitate substantial updates to established protocols, underlying data structures, and key management infrastructures [4, 11, 16]. For instance, ML-KEM (a PQC algorithm used for secure key exchange), even at the AES-128 security level, is significantly larger than X25519 (a non-PQC, elliptic curve Diffie-Hellman key exchange protocol), namely 1,568 bytes vs 64 bytes [16]. This increase can lead to message fragmentation and additional network round-trips within protocols like TLS or DNSSEC, consequently increasing latency and overall data traffic [16]. These larger sizes necessitate significant updates to existing protocols and key management infrastructures [4]. Additionally, the increased data sizes lead to increased electricity consumption for cryptographic operations execution and data storage, which, in turn, negatively affect the sustainability of the information system. Yet, the comparative analysis of PQC implementation in TLS 1.3 (i.e., PQ TLS 1.3) showed that while the specific energy consumption drastically depends on the implementation and system setup (e.g., either implemented unilaterally or mutually), in some cases, PQ TLS 1.3 might be equally or more energy consumption efficient compared to traditional TLS 1.3 [35].

Availability Degradation (PQC.T2). The enlarged size of PQC signatures (such as ML-DSA/Dilithium) may degrade the availability of exchanged messages compared to classical counterparts, particularly in systems with strict message size limitations [11]. However, the NIST standardization process is ongoing, and a specific call for additional signature schemes is currently underway with a focus on short signatures and fast verification. These emerging algorithms aim to mitigate the performance overhead and size-related constraints associated with the first generation of standardized PQC signatures.

Operational Overload (PQC.T3). A large-scale, coordinated transition to PQC could lead to an overload on Certificate Authorities (CAs) as numerous entities simultaneously apply for new PQC certificates, potentially impacting the timely issuance and management of digital identities [16]. To mitigate this threat, Automated Certificate Management Environment (ACME) may be used.

Quantum Key Distribution (QKD) Limitations. While QKD protocols are theoretically secure against quantum attacks, practical implementations may still be vulnerable to classical attacks like **man-in-the-middle (MitM) impersonation (PQC.T4)**, or specific flaws related to equipment defects and channel noise (e.g., PNS Attack, Privacy Amplification) [5]. In case the migration of pure PQC, these threat is not relevant for X-Road. However, it becomes relevant if the quantum key distribution is used.

6.2. Challenges of PQC Implementation

While PQC aims to withstand quantum threats, its implementation introduces new challenges, often resulting in significant costs and extensive timelines [28, 4, 14].

Resource Requirements. PQC algorithms typically demand different and often greater computational resources compared to traditional asymmetric cryptosystems [4, 16]. While some lattice-based signatures can be fast, the overall computational overhead and varying performance across platforms present integration complexities [11, 16].

Infrastructure Updates. Many current hardware appliances designed for cryptographic operations are not compatible with PQC algorithms or their resource demands (e.g., due to the increased key and signature sizes). The examples of such hardware are: (i) *Hardware Security Modules (HSMs)*, dedicated, tamper-resistant physical devices designed to securely generate, store, and manage cryptographic keys, and to perform cryptographic operations; and (ii) *Trusted Platform Modules (TPMs)*, secure crypto processors integrated into a computer's motherboard, and which provide hardware-level security functions, primarily focused on platform integrity and secure storage of cryptographic keys. Consequently, a widespread replacement of such hardware will be required across the enterprise infrastructure [4, 28].

Feasibility and Integration. The practical feasibility of various PQC digital signature algorithms against realworld applications needs thorough analysis and presents integration challenges as they are not extensively researched yet [14]. In particular, ensuring seamless interoperability between existing hybrid systems (combining modern and older technologies) and the new PQC implementations will be a major technical hurdle [4].

Evolving Regulatory Landscape. The landscape of cybersecurity regulations is continuously evolving, with PQC standards actively being developed and mandated by various international and national bodies (e.g., by NIST and BSI) [11, 14, 4]. However, recommendations between these organisations sometimes diverge. This lack of uniformity is often driven by differing institutional security priorities or a preference for specific cryptographic suites aligned with national standards and sovereign requirements [11]. This means that companies have to maintain highly adaptable security architectures to accommodate these varying requirements, which can be complicated and expensive [4].

While the European Commission has not passed a specific law on PQC yet, it has issued a high-level policy document urging all EU countries to work together to protect their digital systems from the threat of quantum computers [36]. This effort is closely linked to eIDAS 2 [37], the updated regulation for electronic identification in the EU. According to a study in [38] and the requirements in eIDAS 2, digital identity service providers must use “current best practices” and “trustworthy implementations” of cryptographic algorithms. This effectively means that new digital identity systems must be built with quantum-resistant standards to ensure they remain secure for the long term.

Interoperability and Compatibility. Ensuring seamless compatibility between new hybrid PQC systems and diverse existing infrastructure during a gradual transition period is a critical challenge [4]. Protecting

data in transit requires mutual agreement between parties on algorithms and formats, as well as robust authentication mechanisms [20]. Additionally, a significant interoperability threat arises from the reliance on Hardware Security Modules (HSMs) and other specialised security hardware. Many current HSMs lack native support for the full range of emerging PQC schemes or are constrained by the significantly larger key and signature sizes (e.g., associated with lattice-based algorithms) [20, 39]. This creates a hardware bottleneck where secure communication may be impossible if one party's hardware foundation cannot process the PQC schemes supported by the other, potentially leading to fragmented security silos during the transition.

Operational Complexity and Maintenance. Some PQC signature schemes (e.g., SPHINCS+) may require more frequent key updates, posing additional operational challenges for key management [11]. Hybrid signature approaches, while secure, require modifications to the system (e.g., Security Server in case of X-Road) to produce and verify two signatures instead of one [20]. However, the latest version of X-Road supports 2 signing algorithms, namely for batch (not eIDAS compliant) or a traditional single message signing (compliant with eIDAS), therefore, the extension of signature creation with PQC option should be relatively easy to implement for extending with a third signing option.

Expertise Gap. A pervasive challenge is the critical shortage of skilled personnel with the necessary expertise in post-quantum cryptography, its implementation, and its secure management [4].

Diversity of PQC Alternatives. Unlike the classical RSA cryptosystem, which can be used for both encryption and signing digital signatures, the first generation of NIST-standardised PQC algorithms consists of specialised schemes for each function (e.g., ML-KEM for key establishment and ML-DSA for signing) [20]. However, this does not represent a lack of functionality; rather, it reflects a shift toward specialized algorithms within a larger pool of cryptographic alternatives. In fact, the PQC landscape offers a broader diversity of mathematical foundations—including lattices, error-correcting codes, and hash-based structures—compared to the narrow reliance on prime factorization and discrete logarithms in traditional cryptography. Furthermore, provably quantum-secure homomorphic encryption algorithms are in the active development phase [19].

Lack of Maturity and Standardisation. While PQC algorithms are under active development and standardisation (e.g., NIST conducts the competition among schemes), their long-term security and performance characteristics are still being rigorously evaluated [6, 14]. There are also differing views on the most conservative PQC schemes, such as those advocated by the German BSI (i.e., Frodo KEM, Classic McEliece), due to the fact that they are based on structured lattices which are, however, relatively new to have been studied and matured by now [11].

6.3. Summary

The migration analysis revealed two key areas of concern. First, there are several new threats enabled by PQC, such as potential side-channel attacks on novel PQC algorithms (PQC.T). Second, there are significant implementation challenges related to the PQC migration, including the increased size of PQC



keys and signatures (impacting network latency and storage) and the operational complexities inherent in rebuilding X-Road's entire Public Key Infrastructure (PKI) to accommodate the new cryptographic standards.

7. Conclusion

This research report set out to investigate the effect of quantum computing and Post-Quantum Cryptography (PQC) on the X-Road data exchange layer. Given the inevitable realisation of fault-tolerant quantum computers and the proven ability of Shor's and Grover's algorithms to compromise the security assumptions of standard (prequantum) cryptography, X-Road, like all digitally dependent infrastructure, faces a definitive and growing quantum threat.

Through a detailed Cryptographic Inventory, this study successfully identified 27 cryptographic assets – keys, hashes, and digital signatures – that are mandatory components for assuring X-Road's fundamental security objectives of confidentiality, integrity, and authenticity. This inventory serves as the baseline for the subsequent threat and risk assessment. Using the inventory, we mapped the 10 identified state-of-the-art quantum computing threats outlined in the literature to specific vulnerabilities within the X-Road ecosystem, detailing the critical QC-enabled risks that must be proactively addressed.

The overarching finding of this report is that the only sustainable defence against the quantum threat is a planned and strategic migration to PQC. To manage the associated risks during this transition, a mitigation strategy was proposed, centered around the implementation of 11 preventative and corrective security controls. These controls, designed specifically to counter the identified QC-enabled risks, must be integrated into X-Road's architectural and operational policies.

At the moment of preparing this report, the implementation costs and from-the-shelf implementations of PQCapable mitigation measures are not known or available yet, e.g., in the form of standardised libraries or protocol implementation. Therefore, the order of mitigation measures in a form of actionable roadmap for X-Road update should be defined in the future work.

Additionally, the results of this study suggest that the shift to PQC is not a final solution but the beginning of a new cryptographic era. The final section highlighted that PQC itself introduces new, novel threats and presents significant implementation challenges. These challenges include operational complexities, standardisation issues, and performance overheads that must be investigated and addressed in the future work to ensure a smooth, and secure migration.

To sum up, X-Road's continued functionality as a trustworthy digital ecosystem relies on prompt and deliberate action. The findings of this report provide the technical roadmap and strategic prerequisites to start PQC migration planning immediately, ensuring that this infrastructure maintains its security posture well into the quantum future.

References

- [1] Nordic Institute for Interoperability Solutions (NIIS), “X-Road Academy.” Available at <https://x-road.thinkific.com/> (visited on 29 October 2024), 2020.
- [2] A. Amadori, T. Attema, M. B. J. D. Duarte, V. Dunning, S. Etinski, D. van Gent, M. Lequesne, W. van der Schoot, and M. Stevens, *The PQC Migration Handbook*. AIVD, TNO and CWI, 2024.
- [3] A. J. Menezes, S. A. Vanstone, and P. C. v. Oorschot, *Handbook of Applied Cryptography*. USA: CRC Press, Inc., 1st ed., 1996.
- [4] K. F. Hasan, L. Simpson, M. A. R. Baee, C. Islam, Z. Rahman, W. Armstrong, P. Gauravaram, and M. McKague, “A framework for migrating to post-quantum cryptography: Security dependency analysis and case studies,” *IEEE Access*, vol. 12, pp. 23427–23450, 2024.
- [5] V. Chamola, A. Jolfaei, V. Chanana, P. Parashari, and V. Hassija, “Information security in the post quantum era for 5G and beyond networks: Threats to existing cryptography, and post-quantum cryptography,” *Computer Communications*, vol. 176, pp. 99–118, 2021.
- [6] A. Kerimbayeva, “Analysis of existing approaches and algorithms of post-quantum cryptography,” *Revue d’Intelligence Artificielle*, vol. 37, no. 3, 2023.
- [7] Y. Qi, J. Bi, L. Li, H. Peng, R. Zhang, and X. He, “A quantum-resistant lightweight hierarchical privacy protection scheme for traffic images,” *IEEE Internet of Things Journal*, pp. 1–1, 2025.
- [8] P. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.
- [9] S. Yevseiev, S. Milevskyi, L. Bortnik, V. Alexey, K. Bondarenko, and S. Pohasii, “Socio-cyber-physical systems security concept,” in *2022 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)*, pp. 1–8, 2022.
- [10] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC ’96, (New York, NY, USA), p. 212–219, Association for Computing Machinery, 1996.
- [11] N. Snetkov and J. Vakarjuk, “Integrating post-quantum cryptography to UXP,” tech. rep., Cybernetica AS, 2023.
- [12] IBM, “Managing cryptography with CBOMkit.” Available at <https://research.ibm.com/blog/quantum-safe-cbomkit> (visited on 20 November 2025), 2024.
- [13] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” Tech. Rep. EBSE-2007-01, 2007.
- [14] R. Shajahan, K. Jain, and P. Krishnan, “A survey on NIST 3rd round post quantum digital signature algorithms,” in *2024 5th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, pp. 132–140, 2024.
- [15] IBM, “Practical introduction to quantum-safe cryptography.” Available: <https://learning.quantum.ibm>.

[com/course/practical-introduction-to-quantum-safe-cryptography](https://www.niis.com/course/practical-introduction-to-quantum-safe-cryptography).

- [16] Bas Westerbaan, “The state of the post-quantum Internet.” Available: <https://blog.cloudflare.com/pq-2024/>, 2024.
- [17] Marin Ivezic, “Shor’s Algorithm: A Quantum Threat to Modern Cryptography.” Available: <https://postquantum.com/post-quantum/shors-algorithm-a-quantum-threat/>, 2017.
- [18] A. Ali, “A pragmatic analysis of pre- and post-quantum cyber security scenarios,” in *2021 International Bhurban Conference on Applied Sciences and Technologies (IBCAST)*, pp. 686–692, 2021.
- [19] L. Gasser, *Post-quantum Cryptography*, pp. 47–52. Cham: Springer Nature Switzerland, 2023.
- [20] J. Vakarjuk, N. Snetkov, and P. Laud, “Identifying obstacles of PQC migration in e-Estonia,” in *2024 16th International Conference on Cyber Conflict: Over the Horizon (CyCon)*, pp. 63–81, 2024.
- [21] National Institute of Standards and Technology, “Post-Quantum Cryptography: Selected Algorithms.” Available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms> (visited on 16 October 2025), 2025.
- [22] R. K. Shashi, M. Manicka Raja, V. More, C. M. Rao, M. Kavitha, and G. Singh, “The intersection of algebra and cryptography: Enhancing information security through mathematical foundations,” *Communications on Applied Nonlinear Analysis*, vol. 31, no. 4s, p. 466 – 489, 2024.
- [23] E. Carozza and G. Couteau, “On threshold signatures from MPC-in-the-head.” Cryptology ePrint Archive, Paper 2024/1897, 2024.
- [24] G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, et al., Status report on the fourth round of the NIST post-quantum cryptography standardization process. US Department of Commerce, National Institute of Standards and Technology, 2025.
- [25] M. Ounsworth, J. Gray, M. Pala, and J. Klaußner, “Composite Signatures For Use In Internet PKI,” InternetDraft draft-ounsworth-pq-composite-sigs-11, Internet Engineering Task Force, 2024. Work in Progress.
- [26] F. Baldimtsi, K. Chalkias, and A. Roy, “Post-quantum readiness in EdDSA chains.” Cryptology ePrint Archive, Paper 2025/1368, 2025.
- [27] J. Proos and C. Zalka, “Shor’s discrete logarithm quantum algorithm for elliptic curves,” *Quantum Info. Comput.*, vol. 3, p. 317–344, July 2003.
- [28] W. Barker, M. Souppaya, and W. Newhouse, “Migration to post-quantum cryptography,” *NIST National Institute of Standards and Technology and National Cybersecurity, Center of Excellence*, pp. 1–15, 2021.
- [29] ETSI, “CYBER; migration strategies and recommendations to quantum safe schemes,” Tech. Rep. ETSI TR 103 619 V1.1.1, 2020.
- [30] DigiCert, “DigiCert Labs.” Available: <https://labs.digicert.com/quantum-safe>, 2024.
- [31] Sectigo®, “Postquantum Cryptography (PQC) Solutions.” Available: <https://www.sectigo.com/quantum-labs>, 2025.

- [32] Bas Westerbaan, “Introducing post-quantum Cloudflare Tunnel.” Available: <https://blog.cloudflare.com/post-quantum-tunnel/>, 2022.
- [33] N. Ricchizzi, C. Schwinne, and J. Pelzl, “Applied post quantum cryptography: A practical approach for generating certificates in industrial environments,” 2025.
- [34] ITU-T, “Recommendation x.509,” Tech. Rep. E 43291, 2019.
- [35] G. Tasopoulos, C. Dimopoulos, A. P. Fournaris, R. K. Zhao, A. Sakzad, and R. Steinfeld, “Energy consumption evaluation of post-quantum tls 1.3 for resource-constrained embedded devices,” in *Proceedings of the 20th ACM International Conference on Computing Frontiers*, CF '23, (New York, NY, USA), p. 366–374, Association for Computing Machinery, 2023.
- [36] European Commission, Directorate-General for Communications Networks, Content and Technology, “Commission Recommendation (EU) 2024/1101 of 11 April 2024 on a Coordinated Implementation Roadmap for the transition to Post-Quantum Cryptography.” Available at <http://data.europa.eu/eli/reco/2024/1101/oj> (visited on 11 August 2025), 2024.
- [37] European Parliament, Council of the European Union, “Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework.” Available at <http://data.europa.eu/eli/reg/2024/1183/oj> (visited on 11 August 2025), 2024.
- [38] G. Comandé and M. Varilek, “The many features which make the eIDAS 2 digital wallet either risky or the ideal vehicle for the transition to post-quantum encryption,” *Computer Law & Security Review*, vol. 54, p. 106022, 2024.
- [39] D. J. Bernstein, A. T. Hülsing, and T. Lange, “Post-quantum cryptography-integration study,” 2022



A. Cryptographic Inventory of X-Road

X-Road Git: <https://github.com/nordic-institute/X-Road> <https://github.com/nordic-institute/X-Road/tree/develop/src>

Tools: [CBOMkit - the essentials for CBOMs](#)

X-Road Cryptography Bill of Materials (CBOM) reports from the CBoM kit:

- CBOMkit_dev_branch_23Apr2025 ([link](#))
- CBOMkit_revision7.6.2_16Jul2025([link](#))

Table 7: Cryptographic assets in the X-Road architecture

ID	Crypto asset type: Name	Function	Protocol	Related X-Road feature	Related components	Source	Threats
Data in transit							
1	Asymmetric Key: Authentication key (and certificate)	Asymmetric encryption	(i) 2048-bit RSA key (by default) (ii) <i>secp256r1</i> curve (EC)	Authentication of Security Servers communication with each other using TLS.	Security Server (stored in soft token)	Link , Link (ECDSA) , Link , Link , Link	T4 on TLS key
2	Encrypted Message: Message encrypted with a	Symmetric encryption (using 2.1)	AES-256 TLS key cypher suites (default value for proxy.xroad-tls-ciphers) [source] :	The messages exchanged between Security Servers are encrypted with a <i>session secret key</i> . The <i>session secret key</i> is established by	Security Server	[22] , Link	



	session secret key		(i)TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (ii) TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (iii) TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384	Security Servers during TLS handshake using Diffie-Hellman key exchange, where Authentication key (key 1) is used for encryption of <i>session secret key</i> .			
2.1	Symmetric Key: Secret Session Key	Symmetric encryption	AES-256 TLS key cypher suites (default value for proxy.xroad-tls-ciphers) [source] : (i)TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (ii) TLS_DHE_RSA_WITH_AES_256_CBC_SHA256	The <i>session secret key</i> is established by Security Servers during TLS handshake using Diffie-Hellman key exchange (ECDHE or DHE), where Authentication key (key 1) is used for encryption of <i>session secret key</i> .	Security Server	[22] , Link	T3, T4



			(iii) TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384				
3	Asymmetric Key: Sign key (and certificate)	Signature	(i) 2048-bit RSA key (by default) (ii) <i>secp256r1</i> curve (EC)	2k, 3k and 4k keys may be simultaneously utilised.	Security Server (stored in soft token or HSM)	Link (ECDSA) Link Link The key generation is implemented here and here .	
4	Hash: Hash of the Certificate Signing Request (CSR)	hash	(i) SHA-256 (ii) SHA-384 (iii) SHA-512	Used for generating a Signing Key and Certificate Signing Request	Security Server	Link , Link	T6
5	Hash: Hash of the message body to be signed	hash	SHA-256	Hash of the SOAP/REST body message to be signed	Security Server	Link	T6
6	Signed Message: Message signed with a sign key	Signature (using 3 and 5)	(i) RSA (by default) (ii) ECDSA		Security Server		T2



7	Hash: hash of certificates for which an OCSP response is requested	hash	SHA-1	The service provider's security server must respond to HTTP GET requests to port 5577 (default configuration). In the HTTP GET request, the client's security server indicates the certificates whose OCSP responses are requested. For this, the client includes cert query parameters whose content is hexadecimally encoded SHA-1 hashes of the certificates.	Security Server (TLS)	link	T6
8	Asymmetric Key: internal (Security Server) SSL/TLS key (and certificate)	Encryption	RSA TLS key cypher suites (default value for proxy.client-tls-ciphers): (i)TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256; (ii)TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256; (iii)TLS_ECDHE_RSA_WITH	Security Server's internal TLS certificate is used in connections between the Security Server and an information system. The internal TLS certificate is used as both a client and a server certificate, depending on the roles of the Security Server and the information system.	Security Server	Link , Link	T4 (to DHE), T3 (to AES), T6 (to MAC)



			H_ AES_256_GCM_SHA384; (iv)TLS_ECDHE_RSA_WIT H_ AES_256_CBC_SHA384; (v) TLS_DHE_RSA_WITH_ AES_128_GCM_SHA256; (vi)TLS_DHE_RSA_WITH_ AES_128_CBC_SHA256; (vii)TLS_DHE_RSA_WITH_ AES_256_CBC_SHA256; (viii)TLS_DHE_RSA_WITH_ _AES_256_GCM_SHA384	The asymmetric key is used for the RSA encryption of the message for establishing a session secret key. The created session secret key is then used for symmetric encryption (using AES-128 or AES-256) of the messages exchanged between Security Servers and its client Information Systems.			
--	--	--	--	--	--	--	--



9	Asymmetric Key: UI/API TLS key (and certificate)	Encryption	2048-bit RSA	The UI/API TLS certificate is used when connecting to the Security Server admin UI or management REST API running in port 4000 (by default). The TLS key and self-signed certificate are auto-generated during the Security Server installation process. The UI/API TLS certificate is generated by the preinst packaging script and the actual generation is implemented by a separate “generate_certificate.sh” script. The OpenSSL configuration used by the script is defined here.	Central Server, Security Server	Link , Link , External Link	
Data at rest							



10	Asymmetric Key: Internal GPG key (per CS or SS)	Encryption or Signature	4096-bit RSA	On the Central Server and the Security Server, this key is used to sign/verify and encrypt/decrypt backup files. On the Security Server, the key is also used to sign and encrypt message log archive files. The private key is used to: - Sign backup files; - Sign message log archive files when message log archive encryption is enabled; - Decrypt backup files during restore The public key is used to: - Encrypt backup files when backup encryption is enabled; - Encrypt message log archive files when message log archive encryption is enabled and archives are not encrypted with a per-member key;	Central Server, Security Server	Link, Link, Link, External Link	
----	--	-------------------------------	--------------	--	------------------------------------	---	--



				- Verify the integrity of a backup file during restore. The internal GPG key is generated by this script.			
11	Signed data: Signed Central Server's backup files	Signature (using 10)	RSA	The Central Server signs/verifies backup files.	Central Server	Link	T2
12	Encrypted data: Encrypted Central Server's backup files	Encryption (using 10)	RSA	The Central Server encrypts/decrypts backup files.	Central Server	Link	T1, T5
13	Asymmetric Key: Backup/restore GPG key	Encryption		Additional public keys that are used to encrypt backup files when backup encryption is enabled. These keys are generated by SS admins and their configuration details depend on the admins. Creating additional keys is explained in the Security Server user guide	Security Server		T3 (if symmetric key), T4 (if with DHE)



14	Encrypted backup: Encrypted Security Server's configuration backup files	Symmetric encryption (using 13)	Internal GPG key is used by default if no additional backup/restore GPG keys have been configured by the SS admin. The details related to those keys apply here as well.	Encrypt Security Server's configuration files when backup encryption is enabled	Security Server	link	T5
15	Key: Message Log Record Encryption Symmetric Keys (for message and attachments)	Symmetric Encryption	16-byte (128-bit) created using HKDF (a simple key derivation function (KDF) based on the HMAC message authentication code) with a SHA-512 hash function	The message log encryption key is generated by the Security Server admin by following the instructions described here . The encryption is symmetric, and the used cypher is AES-CTR.		Link	T3, T4 (if with DHE)
16	Symmetric Encrypted Data: Encrypted (SOAP/REST HTTP) message body in the message log record	Symmetric Encryption (using 15)	AES-128 (Mode of Operation: AES-CTR) [source]	The Security Server encrypts the message's body when it is put in the message log record (before it is appended to the Message Log Database). The Security Server only supports encrypting the message	Security Server	link link	T5



				payload in the message log database.			
17	Hash: Hash of the Message Log record used for the signature	Signature	(at least) SHA-256	Time-stamping authority signature generated a Hash of the Message Log record to be signed.	external TSA	Link	T6
18	Signature: Time-stamping authority signature added to the Message Log record	Signature (using external TSA's key)	(at least) 2048-bit RSA	Timestamped message log records (contain the log of REST and SOAP messages (request/response) and their signatures). The log may contain: (i) <i>full log</i> (The whole message, including both message body and metadata, is logged.); (ii) <i>Metadata logging</i> (Only metadata is logged while the message body is not logged. Verifying the log records afterwards is not possible, and they cannot be used as evidence). The signature details may vary between different TSA	external TSA	Link: 3.10 UC MESS_09 - 3.11 UC MESS_10, Link	T2



				instances, but they must always meet the minimum requirements defined here .			
19	Signed data: Signed Message Log Archive file (ZIP containers of message logs)	Signature (using 10)	RSA	The Security Server periodically composes signed (and timestamped) documents from the (optionally encrypted) message log data and archives them in the local file system. Archive files are ZIP containers containing one or more signed documents and a special linking information file for additional integrity verification purposes. Message log archive encryption and grouping can be enabled and configured separately. The message log archive files are ZIP files that contain ASiC-E (Associated Signature Container	Security Server	Link	T2



				Extended) containers with XADeS signatures. Currently, the implementation is not fully conformant to the technical specification, but that's fixed in X-Road 8. This is the class that implements writing the ZIP files			
20	Key: Message log archive encryption key(s) (per member)	Encryption	Member-specific keys are generated by the Security Server admins, and the configuration details depend on them.	Additional per-member public keys that are used to encrypt message log archive files when message log archive encryption and grouping are enabled. The keys are member-specific and they are used to encrypt archives owned by a specific member.		Creating member-specific keys is explained in the Security Server user guide.	T3 (if symmetric key), T4 (if symmetric key with DHE)
21	Encrypted data: Encrypted Message Log Archive file (ZIP containers of message logs)	Encryption (using 10 or 20)		The Security Server periodically composes signed (and timestamped) documents from the message log data and archives them in the local file system. Archive files are ZIP containers containing one or more	Security Server	Link , Link	T1 (if RSA), T5



				signed documents and a special linking information file for additional integrity verification purposes. Message log archive encryption and grouping can be enabled and configured separately. (optionally) Encrypt message log archive files when message log archive encryption is enabled and archives are not encrypted with a per-member key. The message log archive files are ZIP files that contain ASiC-E (Associated Signature Container Extended) containers with XADeS signatures. Currently, the implementation is not fully conformant to the technical specification, but that's fixed			
--	--	--	--	---	--	--	--



				in X-Road 8. This is the class that implements writing the ZIP files.			
22	Hash: Hash chain of messages in the Message Log database	Hash		The signed in batches messages are connected into a Merkle tree through their hash chain	Security Server	Link Link	T6
Support configurations							
23	Key: Configuration signing key	Asymmetric signature	RSA and EC	The CS administrator generates a configuration source signing key on a security token. The system creates a self-signed certificate containing the public key part of the generated key and generates the configuration anchor containing the created certificate. Used for signing the global configuration (added by an X-Road Security officer).	Central Server (and CS administrator)	Link , Link , Link: 2.2.11 UC GCONF_11	



24	Signature hash algorithm	signature	(i) SHA-512 (by default) (ii) SHA-256, (iii) SHA-384	This is a default signature hash algorithm configured to be used throughout an X-Road instance if custom changes are needed.	X-Road configuration proxy		T6
25	Hash algorithm	hash	(i) SHA256, (ii) SHA512 (by default)	This is a default hash algorithm configured to be used throughout an X-Road instance if custom changes are needed.			T6
26	Hash: Hash of Global Configuration	hash	SHA512	The X-Road governing authority defines the configuration source. The system generates a configuration source, a configuration anchor containing the information needed for the configuration clients to download and verify the configuration from the configuration source.	Central Server & used by Security Server	Link Link	T6



27	Signature for the configuration file: Signature of the Global configuration	Signature (using 23)	RSA (rsa-sha512)	The Global Configuration signature is saved into the Configuration Anchor XML file	Central Server & used by Security Server	Link	T2
----	--	-------------------------	------------------	--	--	----------------------	----

B. Cypher suite in X-Road

- **Crypto Keys:**

- o Sign Key
 - RSA: SHA-256 with RSA [[source](#) central server/admin-service/...authCertif]; Min sign key length = 2048 [[source](#)]
 - EC: *secp256r1* curve [[source](#)]
- o Authentication key:
 - RSA: SHA-256 with RSA [[source](#) central server/admin-service/...authCertif]; Min sign key length = 2048 [[source](#)]
 - EC: *secp256r1* curve [[source](#)];
- o Message log encryption symmetric keys:
 - 16 byte (128bit) created using HKDF [[source](#) X-Road /src/addons/messagelog.../MessageRecordEncryption.java]
 - HKDF (HMAC-based Key Derivation Function) to securely and deterministically generate specific encryption keys from a single master key [[source](#)]
- o Message attachment log encryption symmetric key:
 - 16 byte (128bit) created using HKDF [[source](#) X-Road /src/addons/messagelog.../MessageRecordEncryption.java]
- o Private key & pkcs12 file: [[source](#) X-Road/src/common/common-core .../CertUtils.java]
 - RSA private key [[source](#)] -- private and public keys from PEM file
 - The private key is stored in PKCS#12 file [[source](#)]
- o TLS keys cipher suites [[source1](#), [source2](#)):
 - (proxy.client-tls-ciphers) TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
 - Key Exchange: ECDHE for creating a secure session key with forward secrecy
 - Authentication: RSA for authenticating the server's identity using its digital certificate.
 - Symmetric Encryption: AES_128_GCM for data confidentiality and integrity
 - MAC/Hashing: SHA256 for a hash-based message authentication code
 - (proxy.client-tls-ciphers) TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
 - Key Exchange: ECDHE
 - Authentication: RSA
 - Symmetric Encryption: AES_128_CBC
 - MAC/Hashing: SHA256
 - (proxy.client-tls-ciphers) TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
 - Key Exchange: ECDHE
 - Authentication: RSA
 - Symmetric Encryption: AES_256_GCM
 - MAC/Hashing: SHA384
 - (proxy.client-tls-ciphers) TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
 - Key Exchange: ECDHE
 - Authentication: RSA
 - Symmetric Encryption: AES_256_CBC
 - MAC/Hashing: SHA384
 - (proxy.client-tls-ciphers) TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
 - Key Exchange: DHE for creating a secure session key with forward secrecy.
 - Authentication: RSA



- Symmetric Encryption: AES_128_GCM
- MAC/Hashing: SHA256
- (proxy.client-tls-ciphers) TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
 - Key Exchange: DHE
 - Authentication: RSA
 - Symmetric Encryption: AES_128_CBC
 - MAC/Hashing: SHA256
- (proxy.client-tls-ciphers) TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
 - Key Exchange: DHE
 - Authentication: RSA
 - Symmetric Encryption: AES_256_CBC
 - MAC/Hashing: SHA256
- (proxy.client-tls-ciphers) TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
 - Key Exchange: DHE
 - Authentication: RSA
 - Symmetric Encryption: AES_256_GCM
 - MAC/Hashing: SHA384
- o MAC (secret) key:
 - HMAC_SHA256: used for establishing connection with ACME service using external account binding (EAB) credentials. A secret key (MAC key), usually a base64url-encoded string, used to cryptographically sign a part of the ACME account registration request. This proves that your ACME client is authorized by the external account. [\[source\]](#)
- o External account binding (EAB) credentials
- **Hashes** (i.e. message digest)
 - o SHA512
 - (for hashing message before signing [\[source\]](#))
 - Hash is used to derive a key (using HKDF) for symmetric message log encryption [\[source\]](#)
 - o SHA-256 (saltless password and UUID encoder [\[source\]](#))
- **Algorithm**
 - o **Encryption**
 - AES-CTR [\[source\]](#) X-Road/src/addons/messagelog.../MessageRecordEncryption.java]
 - SHA512 with RSA [\[source\]](#) X-Road/src/common/common-core .../CertUtils.java] – to create a self-signed certificate
 - Message log encryption:
 - AES 128 (Mode of Operation: AES-CTR) [\[source\]](#)
 - HKDF (HMAC-based Key Derivation Function) to securely and deterministically generate specific encryption keys from a single master key. [\[source\]](#)
 - o **Signature**
 - EC: CKM_ECDSA (ECDSA without hashing mechanism [\[source\]](#))
 - RSA: CKM_RSA_PKCS (RSA without hashing [\[source\]](#))

C. Risks to X-Road Enabled by Quantum Computing

Table 8: QC-enabled risks and their components

Business Asset	Crypto Asset	Security criteria*	Threat	Risks	Control
Message	2. Encrypted Message	C	T3. Brute-force attacks on symmetric encryption keys	R1. An attacker executes a brute-force attack on a symmetric secret session key (used in TLS between Security Servers) and identifies the valid secret session key. As a result, an attacker decrypts the messages sent between Security Servers, leading to the loss of message confidentiality.	C7
			T4. Compromise key agreement mechanisms	R2. An attacker compromises the key agreement mechanism for a symmetric secret session key and identifies the valid secret session key. As a result, an attacker decrypts the messages sent between Security Servers, leading to the loss of message confidentiality.	C8
	6. Signed Message	I, Au	T2. Forge digital signatures	R3. An attacker sends a non-original message (completely new or updated from the original) and attaches a forged digital signature of a Member. As a result, the integrity and authenticity of the messages sent from this Member are compromised.	C3, C4, C5, C6



	5. Message Hash	I, C	T6 Quantumenhanced attacks on hash functions	R4. An attacker with access to the message hash executes a quantum-enhanced pre-image attack on the hash function and defines the original message, leading to the loss of message confidentiality. R5. An attacker with access to the message hash executes a quantum-enhanced collision attack, which allows him to find the message with the exact same hash. As a result, the attacker substitutes the original message with the updated one, leading to the loss of message integrity.	C11
	8. Internal SSL/TLS key	I, C	T3 Brute-force attacks on symmetric encryption keys	R6. An attacker executes a brute-force attack on a symmetric Internal SSL/TLS key and identifies the valid Internal SSL/TLS key. As a result, an attacker decrypts the messages sent between the Information System and the Security Server, leading to the loss of message confidentiality.	C7
			T4 Compromise key agreement mechanisms	R7. An attacker compromises the key agreement mechanism for the symmetric part of the Internal SSL/TLS key and identifies the valid Internal SSL/TLS key. As a result, an attacker decrypts the messages sent between the Information System and the Security Server, leading to the loss of message confidentiality.	C8



			T6 Quantum-enhanced attacks on hash functions	R8. An attacker executes a quantum-enhanced pre-image attack on the hash function within the TLS protocol and defines the original message, leading to the loss of message confidentiality. R9. An attacker with access to the hash of the message sent through the TLS protocol executes a quantum-enhanced collision attack, which allows him to find the message with the exact same hash. As a result, the attacker substitutes the original message with the updated one, leading to the loss of message integrity.	C11
	9. UI/API TLS key	C	T4 Compromise key agreement mechanisms	R10. An attacker compromises the key agreement mechanism for the symmetric part of the UI/API TLS key and identifies the valid UI/API TLS key. As a result, an attacker decrypts the messages sent between the Security Server Admin UI and the Security Server, leading to the loss of message confidentiality.	C8
	16. Encrypted Message Body	C	T5 “Harvest Now, Decrypt Later”	R11. An attacker gets access to the encrypted message body stored in the Message log database and stores it. As soon as the attacker obtains quantum-computing powers, he decrypts the harvested earlier message bodies, leading to the loss of message confidentiality.	C1, C9, C10



	15. Message Log Record Encryption Key	C	T3 Brute-force attacks on symmetric encryption keys	R12. An attacker executes a brute-force attack on a symmetric part of the Message Log Record Encryption Key and identifies the valid Message Log Record Encryption Key. As a result, an attacker decrypts the message body stored in the log, leading to the loss of message confidentiality.	C7
		C	T4 Compromise key agreement mechanisms	R13. An attacker compromises the key agreement mechanism for the symmetric part of the Message Log Record Encryption Key and identifies the valid Message Log Record Encryption Key. As a result, an attacker decrypts the message body stored in the log, leading to the loss of message confidentiality.	C8
Message Log	17. Message Log Hash	I, C	T6 Quantum-enhanced attacks on hash functions	R14. An attacker executes a quantum-enhanced pre-image attack on the hash function used for the Message Log Hash and defines the original message log, leading to the loss of message log confidentiality. R15. An attacker with access to the hash of the message log executes a quantum-enhanced collision attack, which allows him to find the message log with the exact same hash. As a result, the attacker substitutes the original message log with the updated one, leading to the loss of message log integrity.	C11



18. Message Log Signature	I, Au	T2 Forge digital signatures	R16. An attacker with access to the message log adds a non-original message log (completely new or updated from the original) and attaches a forged digital signature of a Security Server. As a result, the integrity and authenticity of the message log stored in the Security Server are compromised.	C3, C4, C5, C6
22. Hash chain	I	T6 Quantum-enhanced attacks on hash functions	R17. An attacker with access to the hash chain executes a quantum-enhanced collision attack, which allows him to find the message log with the exact same hash. As a result, the attacker substitutes the original message log with the updated one, leading to the loss of message log integrity.	C11
19. Signed Message Log Archive file	I, Au	T2 Forge digital signatures	R18. An attacker with access to the message log archive adds a non-original message log (completely new or updated from the original) and attaches a forged digital signature of a Security Server. As a result, the integrity and authenticity of the message log stored in the Security Server's Message Log Archive file is compromised.	C3, C4, C5, C6
20. Message log archive encryption key (per Member)	C	T3 Brute-force attacks on symmetric encryption keys	R19. An attacker executes a brute-force attack on a symmetric part of the Message log archive encryption key and identifies the valid Message log archive encryption key. As a result, an attacker decrypts the message log archive stored in the file, leading to	C7



				the loss of message log confidentiality.	
		C	T4 Compromise key agreement mechanisms	R20. An attacker compromises the key agreement mechanism for the symmetric part of the Message log archive encryption key and identifies the valid Message log archive encryption key. As a result, an attacker decrypts the message log stored in the log archive file, leading to the loss of message log confidentiality.	C8
	21. Encrypted Message Archive file	C	T1 Decrypt RSA-encrypted data; T5 "Harvest Now, Decrypt Later"	R21. An attacker gets access to the encrypted message and, using the QC capabilities, decrypts the encrypted message and gets access to the original message log, leading to the loss of message log confidentiality.	C1, C2, C9, C10
Security Server configuration	13. Backup/restore GPG key	C	T3 Brute-force attacks on symmetric encryption keys	R22. An attacker executes a brute-force attack on a symmetric part of the Backup/restore GPG key and identifies the valid Backup/restore GPG key. As a result, an attacker decrypts the Security Server configuration, leading to the loss of Security Server configuration confidentiality.	C7
		C	T4 Compromise key agreement mechanisms	R23. An attacker compromises the key agreement mechanism for the symmetric part of the Security Server configuration and identifies the valid Security Server configuration. As a result, an attacker decrypts the Security Server configuration backup file, leading to the	C8



				loss of Security Server configuration confidentiality.	
	14. Encrypted Security Server's configuration backup files	C	T5 "Harvest Now, Decrypt Later"	R24. An attacker gets access to the encrypted Security Server's configuration backup files and stores them. As soon as the attacker obtains quantum-computing powers, he decrypts the harvested earlier backup files, leading to the loss of Security Server configuration confidentiality.	C1, C9, C10
Global Configuration	26. Hash of a Global Configuration	I, C	T6 Quantum-enhanced attacks on hash functions	R25. An attacker executes a quantum-enhanced pre-image attack on the hash function used for the Hash of a Global Configuration and defines the original Global Configuration, leading to the loss of Global Configuration confidentiality. R26. An attacker with access to the hash of a Global Configuration executes a quantum-enhanced collision attack, which allows him to find the message log with the exact same hash. As a result, the attacker substitutes the original Global Configuration with the updated one, leading to the loss of Global Configuration integrity.	C11
	27. Signature of the Global Configuration	I, Au	T2 Forge digital signatures	R27. An attacker with access to the Global Configuration manipulates the Global Configuration (adding completely new parameters or updating the original ones) and attaches a forged digital signature of a Central Server. As a result, the integrity and authenticity of the Global Configuration	C3, C4, C5, C6



				stored in the XML file are compromised.	
Central Server Configuration	11. Signed Central Server's backup files	I, Au	T2 Forge digital signatures	R28. An attacker with access to the Signed Central Server's backup file manipulates the Central Server configuration (adding completely new parameters or updating the original ones) and attaches a forged digital signature of the Central Server. As a result, the integrity and authenticity of the Central Server Configuration stored in the Signed Central Server's backup files are compromised.	C3, C4, C5, C6
	12. Encrypted Central Server's backup files	C	T1 Decrypt RSA-encrypted data; T5 "Harvest Now, Decrypt Later"	R29. An attacker gets access to the Encrypted Central Server's backup files and, using the QC capabilities, decrypts the encrypted file and gets access to the Central Server Configuration, leading to the loss of the configuration confidentiality.	C1, C2, C9, C10
Member's Sign Key Certificate	4. Hash of the Certificate Signing Request (CSR)	I, Au	T6 Quantum-enhanced attacks on hash functions	R30. An attacker with access to the hash of a Certificate Signing Request (CSR) executes a quantum-enhanced collision attack, which allows him to find the certificate request with the exact same hash. As a result, the attacker substitutes the original Member's Sign Key Certificate CSR with the updated one, leading to the loss of Member's Sign Key Certificate integrity.	C11



	7. Hash of certificates for which an OCSP response is requested	C, I	T6 Quantum-enhanced attacks on hash functions	R31. An attacker executes a quantum-enhanced pre-image attack on the hash function used for the hash of certificates for which an OCSP response is requested and defines the original Member's Sign Key Certificate, leading to the loss of Member's Sign Key Certificate confidentiality. R32. An attacker with access to the hash of certificates for which an OCSP response is requested executes a quantum-enhanced collision attack, which allows him to find the Member's Sign Key Certificate with the exact same hash. As a result, the attacker substitutes the original Member's Sign Key Certificate with the found one, leading to the loss of Member's Sign Key Certificate integrity.	C11
--	---	------	---	---	-----

C - Confidentiality; I - Integrity; Au - Authenticity

D. Risk Assessment Results

Table 9: Assessment of business assets' values

ID	Business Asset (BA)	Business asset value
A1	Message	High (3)
A2	Message Log	High (3)
A3	Security Server Configuration	Low (1)
A4	Global Configuration	Medium (2)
A5	Central Server Configuration	Medium (2)
A6	Member's Sign Key Certificate	Low (1)

Table 10: Assessment of QC-enabled threats likelihood

Threat ID	Threat	Threat Likelihood
T1	Decrypt RSA-encrypted data	Likely (2)
T2	Forge digital signatures	Likely (2)
T3	Brute-force attacks on symmetric encryption keys	Likely (2)
T4	Compromise key agreement mechanisms	Likely (2)
T5	"Harvest Now, Decrypt Later"	Almost Certain (3)
T6	Quantum-enhanced attacks on hash functions	Possible (1)



Table 11: Risk assessment calculation

Risk ID	Affected Assets IDs	Crypto Asset	Affected security criteria	Threat ID	Threat Likelihood ThL	Risk Event Potential EP	Impact level I	Risk Level R	Priority
R1	A1	2. Encrypted Message	C	T3	2	2	3	6	Medium
R2	A1	2. Encrypted Message	C	T4	2	2	3	6	Medium
R3	A1	6. Signed Message	I, Au	T2	2	2	6	12	High
R4	A1	5. Message Hash	C	T6	1	1	3	3	Low
R5	A1	5. Message Hash	I	T6	1	1	3	3	Low
R6	A1	8. Internal SSL/TLS key	C	T3	2	2	3	6	Medium
R7	A1	8. Internal SSL/TLS key	C	T4	2	2	3	6	Medium
R8	A1	8. Internal SSL/TLS key	C	T6	1	1	3	3	Low
R9	A1	8. Internal SSL/TLS key	I	T6	1	1	3	3	Low
R10	A1	9. UI/API TLS key	C	T4	2	2	3	6	Medium
R11	A1	16. Encrypted Message Body	C	T5	3	3	3	9	High
R12	A1	15. Message Log Record Encryption Key	C	T3	2	2	3	6	Medium
R13	A1	15. Message Log Record Encryption Key	C	T4	2	2	3	6	Medium
R14	A2	17. Message Log Hash	C	T6	1	1	3	3	Low
R15	A2	17. Message Log Hash	I	T6	1	1	3	3	Low
R16	A2	18. Message Log Signature	I, Au	T2	2	2	6	12	High
R17	A2	22. Hash chain	I	T6	1	1	3	3	Low
R18	A2	19. Signed Message Log Archive file	I, Au	T2	2	2	6	12	High
R19	A2	20. Message log archive encryption key	C	T3	2	2	3	6	Medium
R20	A2	20. Message log archive encryption key	C	T4	2	2	3	6	Medium
R21	A2	21. Encrypted Message Archive file	C	T1	2	3	3	9	Medium
				T5	3				



Risk ID	Affecte d Assets IDs	Crypto Asset	Affecte d securit y criteria	Threat ID	Threat Likeliho od ThL	Risk Event Potential EP	Impact level I	Risk Level R	Priority
R22	A3	13. Backup/ restore GPG key	C	T3	2	2	1	2	Low
R23	A3	13. Backup/ restore GPG key	C	T4	2	2	1	2	Low
R24	A3	14. Encrypted Security Server's configuration backup files	C	T5	3	3	1	3	Low
R25	A4	26. Hash of a Global Configuration	C	T6	1	1	2	2	Low
R26	A4	26. Hash of a Global Configuration	I	T6	1	1	2	2	Low
R27	A4	27. Signature of the Global Configuration	I, Au	T2	2	2	4	8	Medium
R28	A5	11. Signed Central Server's backup files	I, Au	T2	2	2	4	8	Medium
R29	A5	12. Encrypted Central Server's backup files	C	T1	2	3	2	6	Medium
				T5	3				
R30	A6	4. Hash of the Certificate Signing Request (CSR)	I, Au	T6	1	1	2	2	Low
R31	A6	7. Hash of certificates for which an OCSP response is requested	C	T6	1	1	1	1	Low
R32	A6	7. Hash of certificates for which an OCSP response is requested	I	T6	1	1	1	1	Low